

Qwen3-Next：迈向更极致的训练推理性价比



引言

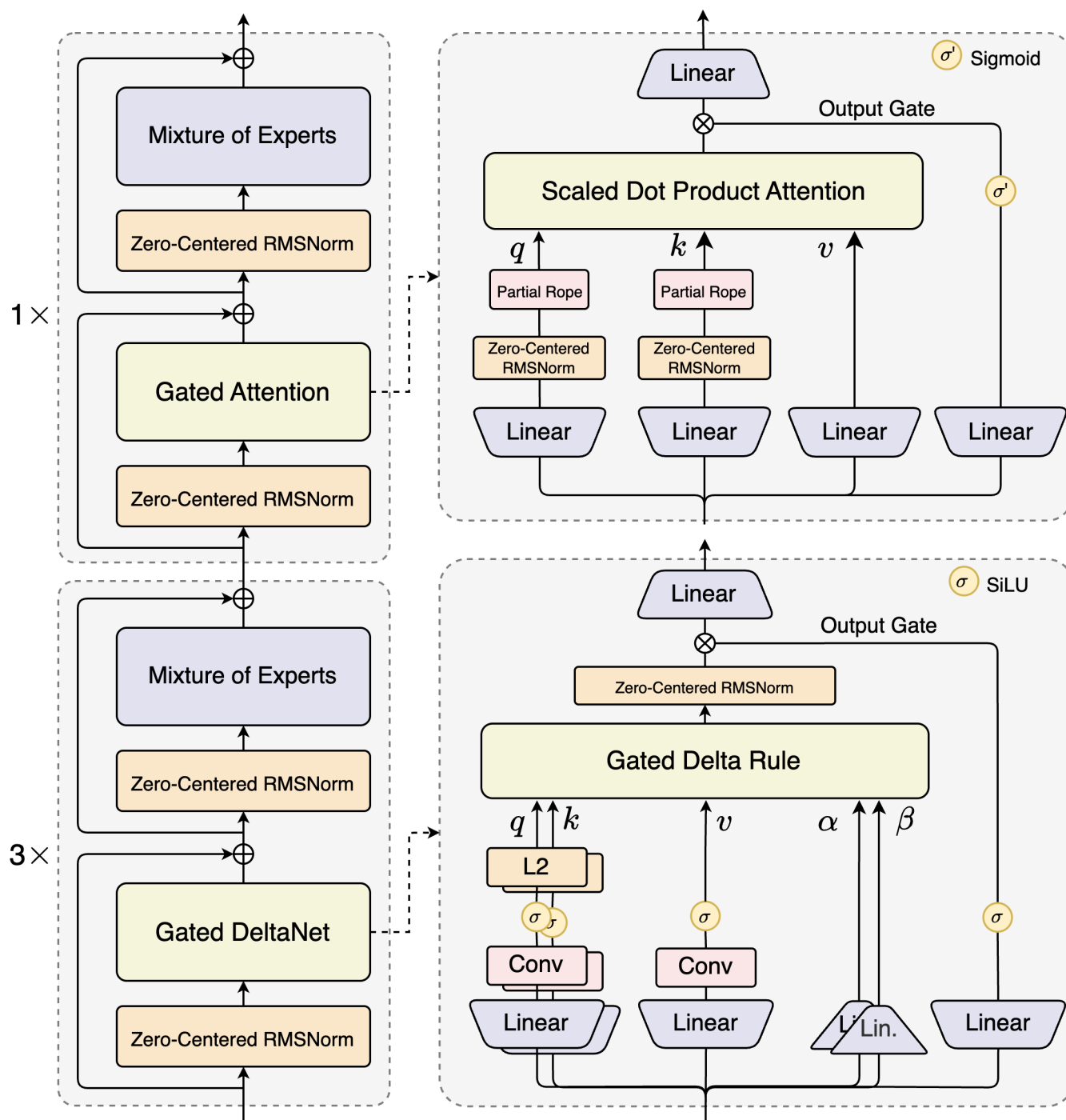
我们认为**Context Length Scaling**和**Total Parameter Scaling**是未来大模型发展的两大趋势，为了进一步提升模型在长上下文和大规模总参数下的训练和推理效率，我们设计了全新的Qwen3-Next的模型结构。该结构相比Qwen3的MoE模型结构，进行了以下核心改进：**混合注意力机制**、**高稀疏度 MoE 结构**、一系列**训练稳定友好的优化**，以及提升推理效率的**多 token 预测机制**。

基于Qwen3-Next的模型结构，我们训练了**Qwen3-Next-80B-A3B-Base**模型，该模型拥有800亿参数仅激活30亿参数。该Base模型实现了与Qwen3-32B dense模型相近甚至略好的性能，而它的训练成本（GPU hours）仅为Qwen3-32B的**十分之一不到**，在32k以上的上下文下的推理吞吐则是Qwen3-32B的**十倍以上**，实现了**极致的训练和推理性价比**。

我们基于 Qwen3-Next-80B-A3B-Base 模型，同步开发并发布了**Qwen3-Next-80B-A3B-Instruct**与 **Qwen3-Next-80B-A3B-Thinking**。我们解决了混合注意力机制 + 高稀疏度 MoE 架构在强化学习训练中长期存在的稳定性与效率难题，实现了 RL 训练效率与最终效果的双重提升。**Qwen3-Next-80B-A3B-Instruct** 与旗舰模型 **Qwen3-235B-A22B-Instruct-2507** 表现相当，同时在 256K 超长上下文处理 任务中展现出显著优势。**Qwen3-Next-80B-A3B-Thinking** 在复杂推理任务上表现卓越，不仅优于预训练成本更高的 **Qwen3-30B-A3B-Thinking-2507** 与 **Qwen3-32B-Thinking**，更在多项基准测试中超越闭源模型 **Gemini-2.5-Flash-Thinking**，部分关键指标已逼近我们Qwen3-235B-A22B-Thinking-2507。

我们已经在 [Hugging Face](#) 和 [ModelScope](#) 开源，同时大家可以在[阿里云百炼](#)以及 [NVIDIA API Catalog](#) 体验 Qwen3-Next 的服务。

模型结构



• 混合架构: Gated DeltaNet + Gated Attention

线性注意力打破了标准注意力的二次复杂度，在处理长上下文时有着更高的效率。我们发现，单纯使用线性注意力或标准注意力均存在局限：前者在长序列建模上效率高但召回能力弱，后者计算开销大、推理不友好。通过系统实验，我们发现 Gated DeltaNet [1] 相比常用的滑动窗口注意力（Sliding Window Attention）和 Mamba2 有更强的上下文学习（in-context learning）能力，并在 3:1 的混合比例（即 75% 层使用 Gated DeltaNet，25% 层保留标准注意力）下能一致超过超越单一架构，实现性能与效率的双重优化。

在保留的标准注意力中，我们进一步引入多项增强设计：（1）沿用我们先前工作 [2] 中的输出门控机制，缓解注意力中的低秩问题。（2）将单个注意力头维度从 128 扩展至 256。（3）仅对注意力头前 25% 的位置维度添加旋转位置编码，提高长度外推效果。

• 极致稀疏 MoE: 仅激活 3.7% 参数

Qwen3-Next 采用了高稀疏度的 Mixture-of-Experts (MoE) 架构，总参数量达80B，每次推理仅激活约 3B 参数。我们实验表明，在使用全局负载均衡 [4] 后，当激活专家固定时，持续增加专家总参数可带来训练 loss 的稳定下降。相比Qwen3 MoE的128个总专家和8个路由专家，Qwen3-Next我们扩展到了512总专家，10路由专家与1共享专家的组合，在不牺牲效果的前提下最大化资源利用率。

• 训练稳定性友好设计

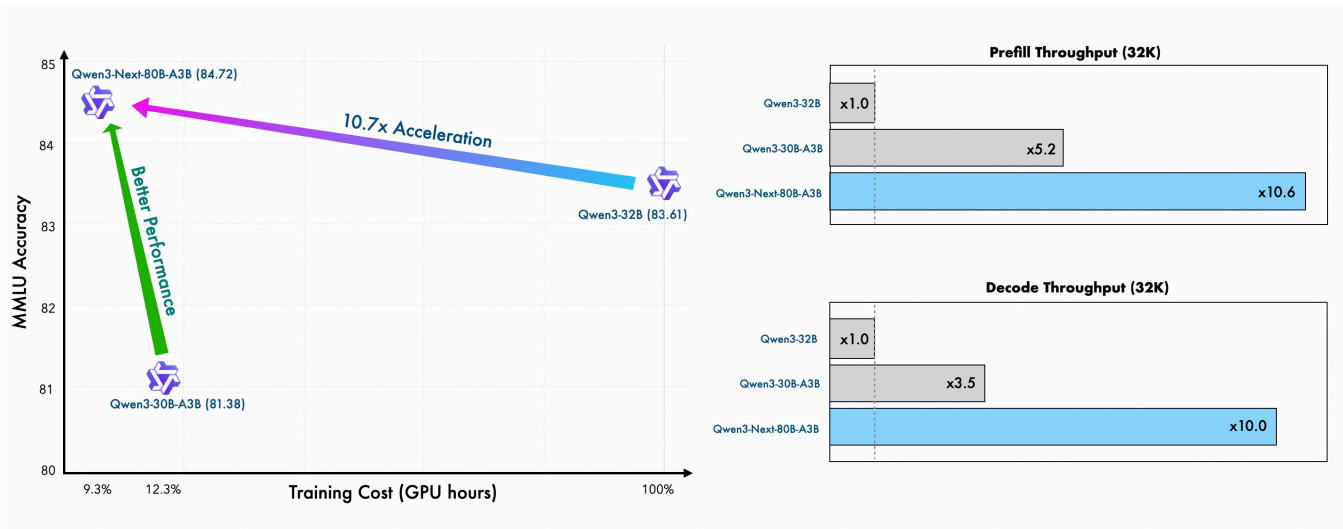
我们发现，注意力输出门控机制能消除注意力池 [5] 与极大激活 [6] 等现象，保证模型各部分的数值稳定。在Qwen3 中我们采用了QK-Norm，我们发现部分层的 norm weight 值会出现异常高的现象。为了缓解这一现象，进一步提高模型的稳定性，我们在Qwen3-Next中采用了 Zero-Centered RMSNorm [7]，并在此基础上，对 norm weight 施加 weight decay，以避免权重无界增长。我们还在初始化时归一化了 MoE router 的参数 [8]，确保每个 expert 在训练早期都能被无偏地选中，减小初始化对实验结果的扰动。

• Multi-Token Prediction

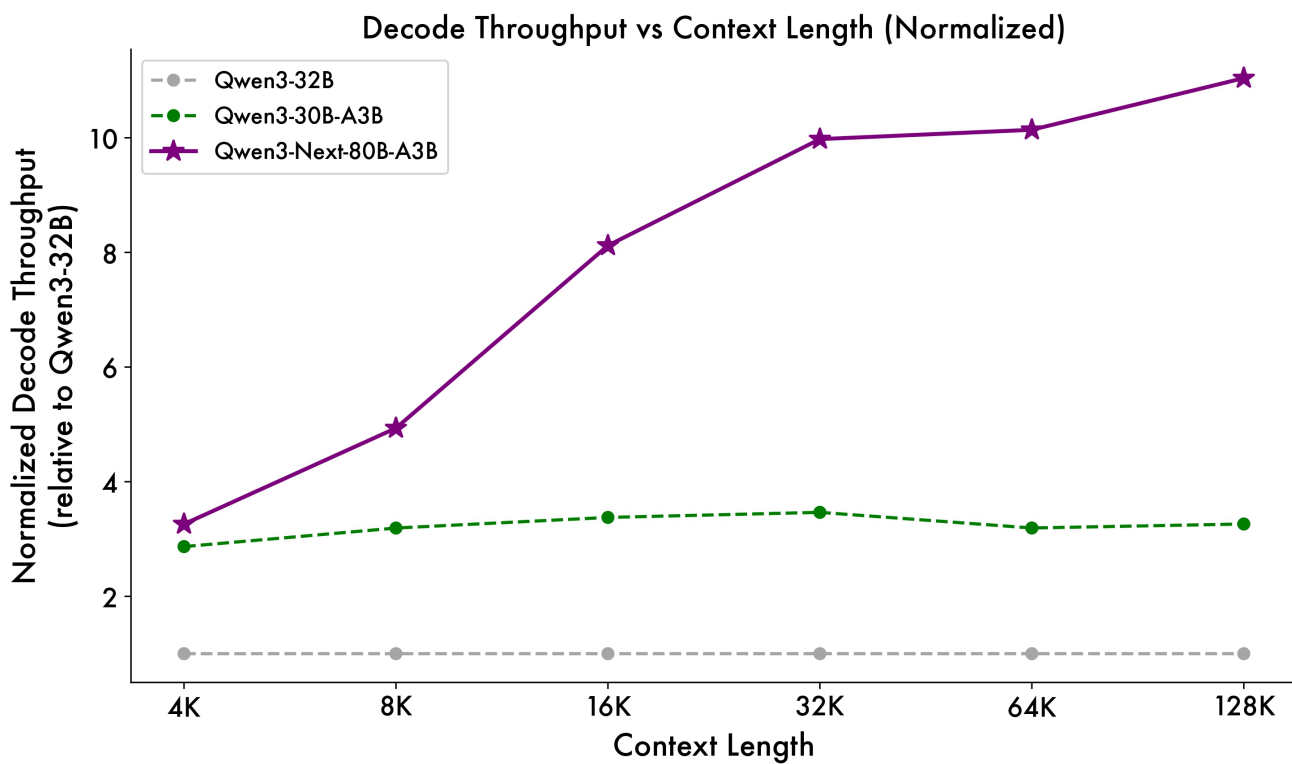
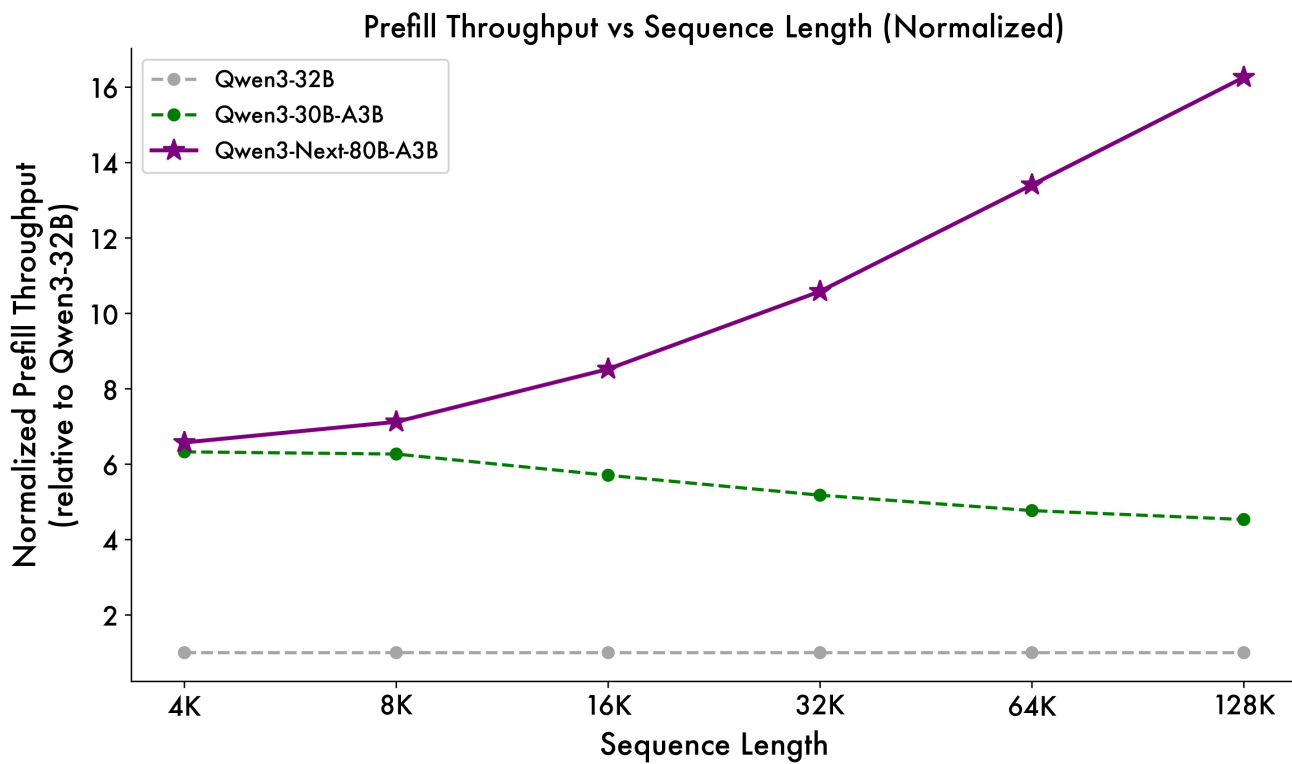
Qwen3-Next 引入原生 Multi-Token Prediction (MTP) 机制 [3][9][10]，既得到了 Speculative Decoding 接受率较高的 MTP 模块，又提升了主干本身的综合性能。Qwen3-Next 还特别优化了 MTP 多步推理性能，通过训练推理一致的多步训练，进一步提高了实用场景下的 Speculative Decoding 接受率。

预训练

• 训练效率及推理效率



Qwen3-Next 采用的是 Qwen3 36T 预训练语料的一个均匀采样子集，仅包含 15T tokens。其训练所消耗的 GPU Hours 不到 Qwen3-30A-3B 的 80%；而与 Qwen3-32B 相比，仅需 9.3% 的 GPU 计算资源，即可实现更优的模型性能，展现出极高的训练效率与性价比。得益于创新的混合模型架构，Qwen3-Next 在推理效率方面表现出显著优势。与 Qwen3-32B 相比，Qwen3-Next-80B-A3B 在预填充（prefill）阶段展现出卓越的吞吐能力：在 4k tokens 的上下文长度下，吞吐量接近前者的七倍；当上下文长度超过 32k 时，吞吐提升更是达到十倍以上。在解码（decode）阶段，该模型同样表现优异——在 4k 上下文下实现近四倍的吞吐提升，而在超过 32k 的长上下文场景中，仍能保持十倍以上吞吐优势。



- **Base模型表现**

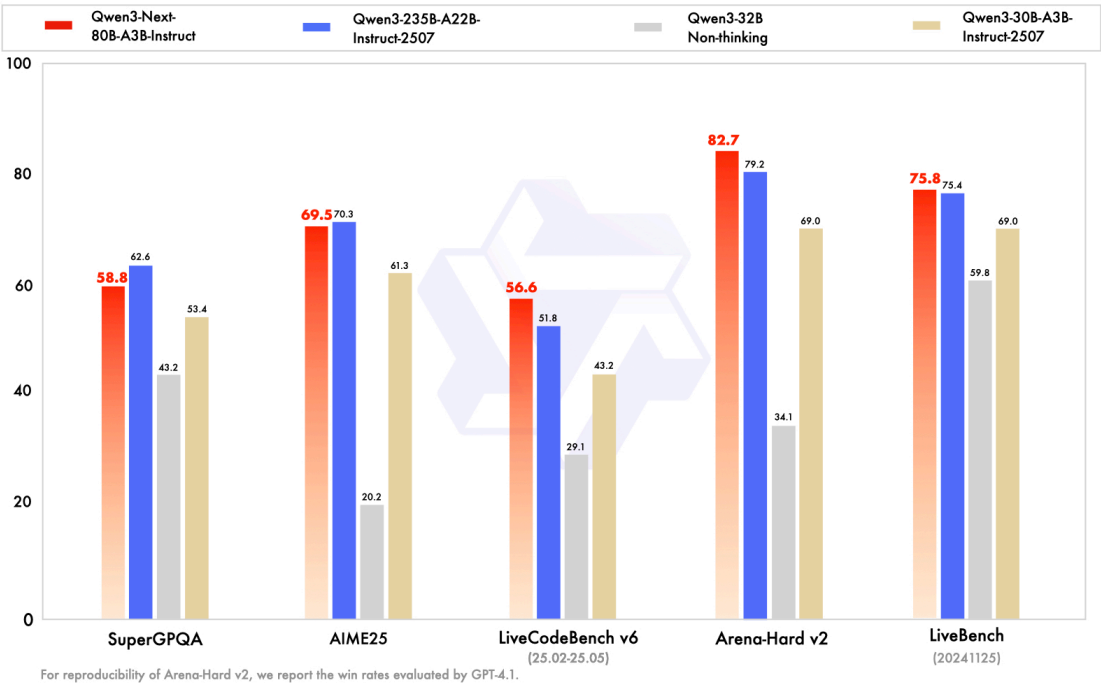
Qwen3-Next-80B-A3B-Base 仅使用十分之一的 Non-Embedding 激活参数，在大多数基准测试中便已超越 Qwen3-32B-Base，且显著优于 Qwen3-30B-A3B，展现出卓越的模型效率与性能优势。

	Qwen3-30B-A3B	Qwen3-32B	Qwen3-Next-80B-A3B	Qwen3-235B-A22B
	Base	Base	Base	Base
Architecture	MoE	Dense	MoE	MoE
# Total Params	30B	32B	80B	235B
# Activated Params	3B	32B	3B	22B
General Tasks				
MMLU	81.38	83.61	<u>84.72</u>	87.81
MMLU-Redux	81.17	83.41	<u>83.80</u>	87.40
MMLU-Pro	61.49	65.54	<u>66.05</u>	68.18
SuperGPQA	35.72	39.78	<u>41.52</u>	44.06
BBH	81.54	<u>87.38</u>	87.13	88.87
Math, STEM & Coding Tasks				
GPQA	43.94	49.49	43.43	<u>47.47</u>
GSM8K	91.81	<u>93.40</u>	90.30	94.39
MATH	59.04	61.62	62.36	71.84
EvalPlus	71.45	72.05	<u>72.89</u>	77.60
CRUX-O	67.20	72.50	<u>74.25</u>	79.00
Multilingual Tasks				
MGSM	79.11	<u>83.06</u>	81.28	83.53
MMMLU	81.46	83.83	<u>84.43</u>	86.70
INCLUDE	67.00	67.87	<u>69.79</u>	73.46

后训练

- Instruct模型表现

Qwen3-Next-80B-A3B-Instruct 显著优于 Qwen3-30B-A3B-Instruct-2507 和 Qwen3-32B-Non-thinking，并取得了几乎与 Qwen3-235B-A22B-Instruct-2507 相近的结果。

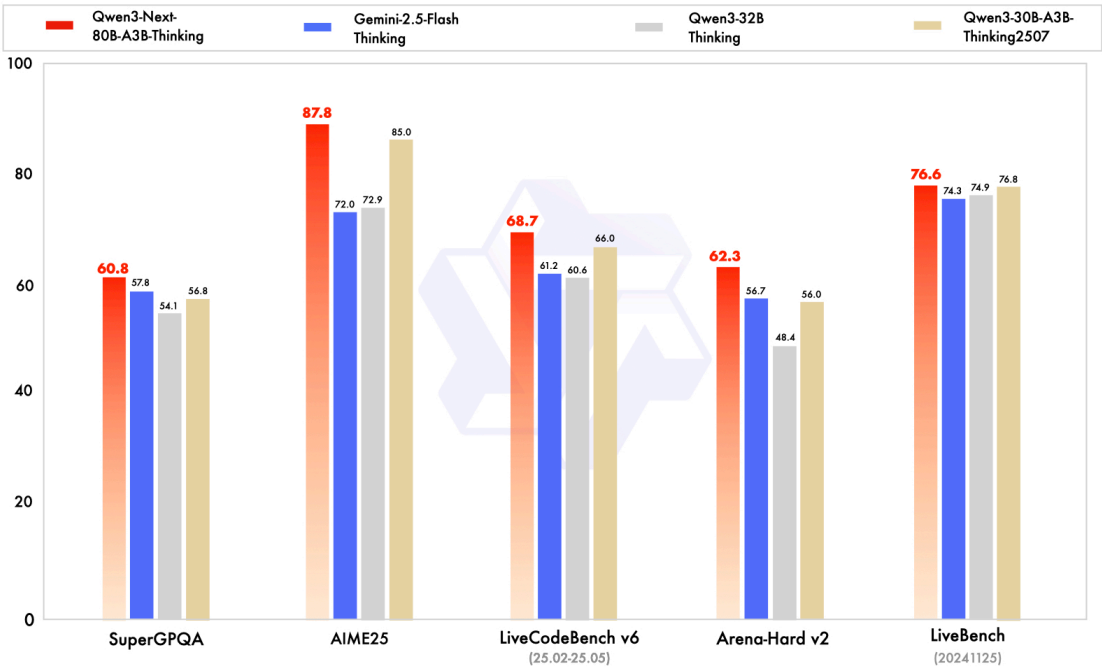


Qwen3-Next-80B-A3B-Instruct 在RULER上所有长度的表现明显优于层数相同、注意力层数更多的 Qwen3-30B-A3B-Instruct-2507，甚至在 256k 范围内都超过了层数更多的 Qwen3-235B-A22B-Instruct-2507，这展示了 Gated DeltaNet 与 Gated Attention 混合模型在长文本情景下的优越性。

Model	RULER															
	Avg.	4K	8K	16K	32K	64K	96k	128K	192k	256k	384k	512k	640k	768k	896k	1M
Qwen3-30B-A3B-Instruct-2507	86.8	98.0	96.7	96.9	97.2	93.4	91.0	89.1	89.8	82.5	83.6	78.4	79.7	77.6	75.7	72.8
Qwen3-235B-A22B-Instruct-2507	92.5	98.5	97.6	96.9	97.3	95.8	94.9	93.9	94.5	91.0	92.2	90.9	87.8	84.8	86.5	84.5
Qwen3-Next-80B-A3B-Instruct	91.8	98.5	99.0	98.0	98.7	97.6	95.0	96.0	94.0	93.5	91.7	86.9	85.5	81.7	80.3	80.3

• Thinking模型表现

Qwen3-Next-80B-A3B-Thinking 优于预训练成本更高的 Qwen3-30B-A3B-Thinking-2507 和 Qwen3-32B-thinking，超过了闭源的模型 Gemini-2.5-Flash-Thinking，并在部分指标上接近了我们的最新的旗舰模型 Qwen3-235B-A22B-Thinking-2507。



开始使用 Qwen3

以下示例均基于 Qwen3-Next-80B-A3B-Instruct 版本给出，Thinking 模型请参考<https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Thinking>。

Hugging Face Transformers

Qwen3-Next 的代码已合并至 Hugging Face `transformers` 的主分支。

```
pip install git+https://github.com/huggingface/transformers.git@main
```

若使用较早版本，您将遇到以下错误：

```
KeyError: 'qwen3_next'
```

下方代码片段演示了如何基于给定输入使用模型生成内容：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "Qwen/Qwen3-Next-80B-A3B-Instruct"

# load the tokenizer and the model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    dtype="auto",
    device_map="auto",
)

# prepare the model input
prompt = "Give me a short introduction to large language model."
messages = [
    {"role": "user", "content": prompt},
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

# conduct text completion
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=16384,
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()

content = tokenizer.decode(output_ids, skip_special_tokens=True)

print("content:", content)
```

Note

- Hugging Face Transformers 目前尚未普遍支持 Multi-Token Prediction (MTP)。

Note

- 效率或吞吐量提升高度依赖具体实现。
- 建议采用专用推理框架（如 SGLang 和 vLLM）执行推理任务。

Tip

-
- 根据推理设置，您可能通过安装 [flash-linear-attention](#) 和 [causal-conv1d](#) 获得更佳效率。
-
- 请参阅上述链接获取详细安装说明和依赖要求。

部署时，您可以使用最新的 `sglang` 或 `vllm` 创建兼容 OpenAI 的 API 接口。

SGLang

[SGLang](#) 是一个面向大语言模型与视觉语言模型的高性能服务框架，可用于启动兼容 OpenAI API 的服务。

SGLang 已在其 `main` 分支中支持 Qwen3-Next，可通过源码安装：

```
pip install 'sglang[all] @ git+https://github.com/sgl-project/sglang.git@main#subdirectory=python'
```

以下命令可在 4 个 GPU 上使用张量并行，创建最大上下文长度为 256K token 的 API 接口（地址：`http://localhost:30000/v1`）：

```
SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1 python -m sglang.launch_server --model-path Qwen/Qwen3-Next-80B-A3B-Instruct --port 30000 --tp-size 4 --context-length 262144 --mem-fraction-static 0.8
```

若要启用 MTP（其余设置同上），推荐使用以下命令：

```
SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1 python -m sglang.launch_server --model-path Qwen/Qwen3-Next-80B-A3B-Instruct --port 30000 --tp-size 4 --context-length 262144 --mem-fraction-static 0.8 --speculative-algo NEXTN --speculative-num-steps 3 --speculative-eagle-topk 1 --speculative-num-draft-tokens 4
```

Note

-
- 当前必须设置环境变量 `SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1`。

Note

-
- 默认上下文长度为 256K。若服务启动失败，建议将上下文长度减小至更小值，例如 `32768`。

vLLM

[vLLM](#) 是一个高吞吐量、高内存效率的大语言模型推理与服务引擎，可用于启动兼容 OpenAI API 的服务。

vLLM 已在其 `main` 分支中支持 Qwen3-Next，可通过源码安装：

```
pip install vllm --pre --extra-index-url https://wheels.vllm.ai/nightly
```

以下命令可在 4 个 GPU 上使用张量并行，创建最大上下文长度为 256K token 的 API 端点（地址：`http://localhost:8000/v1`）：

```
VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm serve Qwen/Qwen3-Next-80B-A3B-Instruct --port 8000 --tensor-parallel-size 4 --max-model-len 262144
```

若要启用 MTP（其余设置同上），推荐使用以下命令：

```
VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm serve Qwen/Qwen3-Next-80B-A3B-Instruct --port 8000 --
tensor-parallel-size 4 --max-model-len 262144 --speculative-config
'{"method":"qwen3_next_mtp","num_speculative_tokens":2}'
```

Note

-
- 当前必须设置环境变量 `VLLM_ALLOW_LONG_MAX_MODEL_LEN=1`。

Note

-
- 默认上下文长度为 256K。若服务启动失败，建议将上下文长度减小至更小值，例如 `32768`。

Agentic Use

Qwen3 在工具调用能力方面表现卓越。我们推荐使用 [Qwen-Agent](#) 以充分发挥 Qwen3 的智能体能力。Qwen-Agent 内部封装了工具调用模板与解析器，可大幅降低编码复杂度。

您可通过 MCP 配置文件、Qwen-Agent 内置工具或自行集成其他工具来定义可用工具

```
from qwen_agent.agents import Assistant

# Define LLM
llm_cfg = {
    'model': 'Qwen3-Next-80B-A3B-Instruct',

    # Use a custom endpoint compatible with OpenAI API:
    'model_server': 'http://localhost:8000/v1', # api_base
    'api_key': 'EMPTY',
}

# Define Tools
tools = [
    {'mcpServers': { # You can specify the MCP configuration file
        'time': {
            'command': 'uvx',
            'args': ['mcp-server-time', '--local-timezone=Asia/Shanghai']
        },
        "fetch": {
            "command": "uvx",
            "args": ["mcp-server-fetch"]
        }
    }
},
    'code_interpreter', # Built-in tools
]

# Define Agent
bot = Assistant(llm=llm_cfg, function_list=tools)

# Streaming generation
```

```
messages = [{'role': 'user', 'content': 'https://qwenlm.github.io/blog/ Introduce the latest  
developments of Qwen'}]  
for responses in bot.run(messages=messages):  
    pass  
print(responses)
```

处理超长上下文

Qwen3-Next 原生支持高达 262,144 个 token 的上下文长度。对于总长度（输入+输出）显著超过此限制的对话，我们推荐使用 RoPE 缩放技术高效处理长文本。我们已使用 [YaRN](#) 方法验证模型在高达 100 万 token 上下文长度下的性能。

目前，多个开源推理框架（如 `transformers`、`vllm` 和 `sglang`）均已支持 YaRN。启用 YaRN 通常有两种方式：

- 修改模型文件：
在 `config.json` 文件中添加 `rope_scaling` 字段：

```
{  
    ...,  
    "rope_scaling": {  
        "rope_type": "yarn",  
        "factor": 4.0,  
        "original_max_position_embeddings": 262144  
    }  
}
```

- 通过命令行参数传递：对于 `vllm`，可使用：

```
VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm serve ... --rope-scaling  
'{"rope_type":"yarn","factor":4.0,"original_max_position_embeddings":262144}' --max-model-  
len 1010000
```

对于 `sglang`，可使用：

```
SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1 python -m sglang.launch_server ... --json-model-  
override-args '{"rope_scaling":  
{"rope_type":"yarn","factor":4.0,"original_max_position_embeddings":262144}}' --context-  
length 1010000
```

Note

- 所有主流开源框架目前实现的均为静态 YaRN，即缩放因子不随输入长度变化，**这可能影响短文本的性能**。
- 我们建议仅在需要处理长上下文时才添加 `rope_scaling` 配置。
- 同时，建议根据实际需求调整 `factor` 参数。例如，若您的应用场景典型上下文长度为 524,288 token，则设置 `factor` 为 2.0 更为合适。

总结

Qwen3-Next在模型架构上实现了重大突破，引入了注意力机制方面的多项创新，包括线性注意力和注意力门控机制，并进一步提高了MoE的稀疏度。Qwen3-Next-80B-A3B在思考模式和非思考模式下的性能均与规模更大的Qwen3-235B-A22B-2507相当，同时在推理速度上，特别是长上下文场景中，提升显著。此次发布，我们旨在赋能开源社区，使其能够与前沿架构创新同步演进。展望未来，我们将持续优化这一架构，开发Qwen3.5，致力于实现更高的智能水平与生产力。

参考文献

- 1.Gated Delta Networks: Improving Mamba2 with Delta Rule
- 2.Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free
- 3.DeepSeek-V3 Technical Report
- 4.Demons in the Detail: On Implementing Load Balancing Loss for Training Specialized Mixture-of-Expert Models
- 5.Efficient Streaming Language Models with Attention Sinks
- 6.Massive Activations in Large Language Models
- 7.Gemma: Open Models Based on Gemini Research and Technology
- 8.Approximating Two-Layer Feedforward Networks for Efficient Transformers
- 9.Better & faster large language models via multi-token prediction
- 10.ProphetNet: Predicting Future N-gram for Sequence-to-Sequence Pre-training